

AGOD Internal Project & Payout Tracker

Advanced System Design and Recommended Technology Stack

Organization: AGOD Software Solutions (AlphaGroupOfDevelopers)

Document status: Recommended implementation baseline

Audience: Founder/Admin, Project Managers, engineering team

Supersedes: AGOD_Project_Payout_Tracker_System_Design(1)

Recommended first release: Internal MVP for approximately 10 team members

1. Executive recommendation

Build this as a small, secure internal web application with a **modular monolith** architecture rather than microservices.

Recommended stack

Layer	Recommendation	Why
Frontend and application	Next.js + TypeScript	One codebase for the internal dashboard, server-side authorization, routing, and API/server actions
UI	Tailwind CSS + shadcn/ui	Fast, consistent internal-tool interface with accessible primitives
Forms and validation	React Hook Form + Zod	Strong client/server validation for money, splits, dates, and state transitions
Database	PostgreSQL via Supabase	Relational integrity, transactions, backups, SQL reporting, and a good fit for ledger-style data
Authentication	Supabase Auth or the existing Ledgio auth integration	Avoid a second identity system if Ledgio already provides a reliable internal login
Authorization	Application permission checks + PostgreSQL Row Level Security	Defense in depth; users should only see and mutate what their role permits
File storage	Supabase Storage (optional in MVP)	Store approval evidence, invoices, receipts, or payment proof without putting files in the database
Notifications	In-app notifications first; email/WhatsApp later	Keep MVP simple and avoid operational dependencies before the workflow is proven
Deployment	Vercel for the app + Supabase for data/auth	Minimal operations for a small internal team
Observability	Sentry + structured application logs	Catch authorization, workflow, and production errors early

Layer	Recommendation	Why
Testing	Vitest, Testing Library, Playwright	Unit-test payout calculations and state rules; use a few end-to-end workflow tests
Source control/CI	GitHub + GitHub Actions	Pull-request review, migrations, tests, and deployment checks

Key architectural decision

Use a **modular monolith** with clear modules:

- Identity and access
- Projects
- Assignments and compensation plans
- Tasks and milestones
- Approvals
- Payout ledger
- Manual payments and reconciliation
- Audit log
- Dashboard/reporting

This is easier to ship and maintain than separate services for a team of ten. The system should still expose clean module boundaries so a payment integration or accounting export can be added later.

Do not start with automated MoMo or bank payments. First make the calculation, approval, audit, and reconciliation process trustworthy.

2. Review of the original brief

The original brief has a strong foundation:

- It correctly limits the first release to internal use.
- It keeps actual money movement outside the system.
- It identifies approval as the trigger for payable amounts.

- It includes a payout ledger and audit trail.
- It avoids hourly billing and client-facing complexity.

The following changes are needed before implementation:

2.1 Separate project status from payout status

The original flow ends with a project status of `Paid`. That does not work when:

- one contributor is paid before another;
- a payout is partial;
- a project has an adjustment;
- a project is completed but payment is delayed;
- a project has no payout because the split was zero or the work was voluntary.

Use separate state machines:

Project status

```
DRAFT → PLANNING → IN_PROGRESS → PENDING_APPROVAL → COMPLETED
                                     ↘ REJECTED / CHANGES_REQUESTED
```

Payout status per recipient

```
NOT_CREATED → OWED → PARTIALLY_PAID → PAID
                                   ↘ DISPUTED / VOIDED
```

The dashboard can still show a derived project payout state such as `Unpaid`, `Partially paid`, or `Fully paid`.

2.2 Freeze the compensation plan at approval

Assignments and split percentages must be editable while a project is being planned, but the system must create an immutable **compensation snapshot** when the project is approved.

That snapshot should store:

- project value and currency;

- each recipient;
- split type and value;
- calculated amount;
- rounding result;
- role on project;
- rationale;
- the user and timestamp that approved it.

If a correction is needed after approval, create a controlled adjustment rather than silently changing the original record.

2.3 Model payments as transactions, not a single paid flag

A single `paid_by` and `paid_date` field cannot represent partial payments or multiple payment methods. Add a `payment_transactions` table for manual payment confirmations.

The ledger entry remains the amount owed; payment transactions explain how much has actually been paid.

2.4 Define money and currency rules

The system should not use floating-point numbers for GHS amounts. Store money as integer minor units, for example pesewas, together with an ISO currency code.

Example:

```
amount_minor = 125000  
currency = GHS
```

For percentages, store a fixed decimal scale, such as basis points or `numeric(7,4)`, and validate totals before approval.

2.5 Clarify task completion versus project approval

A contributor marking a task `Done` is evidence of readiness, not approval. The project should be eligible for approval only when:

- required tasks are complete or explicitly waived;
- the compensation plan is valid;

- the project value and currency are present;
- the approver has reviewed the completion evidence.

A PM/Admin may override an incomplete task only with a required reason recorded in the audit log.

3. Scope

3.1 In scope for MVP

1. Secure internal login.
2. Role-based access for Team Member, Project Manager, and Admin.
3. Project creation and editing before approval.
4. Project assignments and compensation splits.
5. Milestones and tasks.
6. Project completion request.
7. PM/Admin approval or rejection with reason.
8. Immutable compensation snapshot at approval.
9. Payout ledger generated from the snapshot.
10. Manual payment recording, including partial payments.
11. Search, filters, and dashboards.
12. Complete audit history for material actions.
13. CSV export for bookkeeping/reconciliation.
14. Basic backup, error monitoring, and deployment checks.

3.2 Explicitly out of scope for MVP

- Automated bank or Mobile Money transfers.
- Client-facing access.
- Full accounting, tax, payroll, or invoicing functionality.
- Hourly timesheets.
- Complex multi-stage approval chains.
- Currency conversion.

- Public API for third parties.
- Automatic WhatsApp messaging.

4. Roles and permissions

Capability	Team Member	Project Manager	Admin
View assigned projects/tasks	Yes	Yes	Yes
Update own task status	Yes	Yes	Yes
Create projects	No	Yes	Yes
Edit project before approval	No	Yes	Yes
Configure assignments and splits	No	Yes	Yes
Request project approval	Yes, for assigned work	Yes	Yes
Approve/reject completion	No	Yes	Yes
View own payout records	Yes	Yes	Yes
View all payout records	No	Yes	Yes
Record manual payment	No	Optional	Yes
Create adjustment/void payout	No	No or controlled	Yes
Manage team members and roles	No	No	Yes
Export ledger	No	Yes	Yes
View full audit log	No	Limited project scope	Yes

Authorization rules

- Permissions must be enforced server-side, not only by hiding UI controls.

- A team member can update only their own assigned task records unless explicitly assigned a project-lead permission.
 - A PM cannot approve their own payout adjustment without an Admin review if AGOD wants separation of duties.
 - Admin access should be rare and audited.
 - Inactive members remain visible in historical records but cannot receive new assignments unless reactivated.
-

5. Core workflow

5.1 Project lifecycle

1. **Create project** with name, client type, project value, currency, dates, and project owner.
2. **Configure compensation plan** by assigning members and adding percentage or fixed splits.
3. **Validate plan**:
 - percentage splits total 100% when percentage-only;
 - fixed splits do not exceed project value;
 - mixed plans follow an explicit rule;
 - no duplicate active assignment for the same member and role;
 - required rationale is present for overrides.
4. **Create milestones/tasks** and assign responsibility.
5. **Track execution** through task and milestone statuses.
6. **Request approval** when required work is complete.
7. **Approve or reject**:
 - approval freezes the compensation snapshot and creates payout ledger entries in one database transaction;
 - rejection returns the project to **In Progress** with a reason.
8. **Record manual payments** as one or more payment transactions.
9. **Reconcile** total payments against each ledger entry.
10. **Close and report** through dashboard and CSV export.

5.2 Approval transaction

The approval action must be atomic:

```
BEGIN
  lock project row
  verify user is PM/Admin
  verify project is PENDING_APPROVAL
  validate required tasks and compensation plan
  create compensation_snapshot
  create payout_ledger_entries
  append audit event
  update project to COMPLETED
COMMIT
```

If any step fails, none of the payout records should be created.

5.3 Rejection and reopening

A rejection must require a reason. The project returns to `IN_PROGRESS` or `CHANGES_REQUESTED`. The system should notify the project owner in-app.

After approval, the project should be locked for ordinary edits. An Admin can reopen it only through a controlled action requiring a reason and an audit event.

6. Recommended data model

6.1 Main entities

users

- `id` UUID primary key
- `name`
- `email`
- `phone` nullable
- `role` enum: `TEAM_MEMBER`, `PROJECT_MANAGER`, `ADMIN`
- `active` boolean

- `created_at` , `updated_at`

projects

- `id` UUID primary key
- `code` human-readable identifier, e.g. `AGOD-2026-001`
- `name`
- `description` nullable
- `client_type` enum: `INTERNAL` , `EXTERNAL`
- `client_name` nullable
- `total_value_minor` integer
- `currency` char(3), default `GHS`
- `status` enum: `DRAFT` , `PLANNING` , `IN_PROGRESS` , `PENDING_APPROVAL` , `COMPLETED` , `CANCELLED`
- `project_owner_id` foreign key to users
- `start_date` , `target_date` , `completed_at` nullable
- `created_by` , `created_at` , `updated_at`
- `approved_by` , `approved_at` nullable
- `version` integer for optimistic concurrency

project_assignments

- `id` UUID primary key
- `project_id`
- `member_id`
- `role_on_project`
- `split_type` enum: `PERCENTAGE` , `FIXED_AMOUNT`
- `split_value` numeric or integer minor units depending on type
- `rationale` nullable
- `active` boolean
- `created_at` , `updated_at`

Add a unique constraint preventing duplicate active assignments for the same project/member/role where appropriate.

milestones

- id
- project_id
- title
- description nullable
- sequence
- due_date nullable
- status: NOT_STARTED, IN_PROGRESS, COMPLETED
- completed_at nullable

tasks

- id
- milestone_id nullable
- project_id
- title
- description nullable
- assigned_to
- status: NOT_STARTED, IN_PROGRESS, DONE, WAIVED
- due_date, completed_at
- completion_note nullable
- completed_by nullable

compensation_snapshots

- id
 - project_id unique
 - project_total_value_minor
 - currency
 - created_by
 - created_at
 - calculation_version
-

- `calculation_notes` nullable

compensation_snapshot_lines

- `id`
- `snapshot_id`
- `member_id`
- `role_on_project`
- `source_assignment_id`
- `split_type`
- `split_value`
- `amount_owed_minor`
- `currency`
- `rationale` nullable

These rows are immutable after approval.

payout_ledger_entries

- `id`
- `project_id`
- `snapshot_line_id`
- `member_id`
- `amount_owed_minor`
- `amount_paid_minor` derived from payment transactions or maintained transactionally
- `currency`
- `status`: `OWED`, `PARTIALLY_PAID`, `PAID`, `DISPUTED`, `VOIDED`
- `approved_by`, `approved_at`
- `notes` nullable
- `created_at`, `updated_at`

Use a database check or service rule to ensure `amount_paid_minor ≤ amount_owed_minor` unless an Admin records a separately approved adjustment.

payment_transactions

- id
- ledger_entry_id
- amount_minor
- currency
- method : MOBILE_MONEY , BANK_TRANSFER , CASH , OTHER
- reference nullable
- paid_at
- recorded_by
- evidence_file_path nullable
- notes nullable
- created_at

adjustments

- id
- ledger_entry_id
- type : INCREASE , DECREASE , WRITE_OFF , VOID
- amount_minor
- reason
- created_by
- approved_by nullable
- created_at

Adjustments should never overwrite the original amount. They create a visible correction trail.

audit_events

- id
- actor_id
- entity_type
- entity_id
- action

- `before_json` nullable
- `after_json` nullable
- `reason` nullable
- `ip_hash` nullable
- `user_agent` nullable
- `created_at`

Audit events should be append-only for normal application users.

notifications

- `id`
- `recipient_id`
- `type`
- `title`
- `message`
- `entity_type` , `entity_id`
- `read_at` nullable
- `created_at`

6.2 Important constraints

- All foreign keys should use explicit delete behavior. Avoid cascading deletion of financial history.
 - Prefer soft deletion or archival for projects, users, assignments, and ledger records.
 - A project cannot be approved without a valid compensation snapshot.
 - A payout cannot be marked fully paid by editing a status field; it must be supported by payment transactions.
 - Every approval, rejection, payment, adjustment, reopening, and role change must generate an audit event.
 - Store timestamps in UTC and render them in AGOD's operating timezone.
-

7. Split calculation rules

Recommended MVP rule: do not mix split types within one project

Allow either:

- **Percentage mode:** all active assignments use percentages totaling exactly 100%; or
- **Fixed mode:** all active assignments use fixed amounts totaling no more than the project value.

Mixed percentage/fixed plans can be added later, but they create ambiguity about the order of calculation and who absorbs rounding differences.

Percentage calculation

```
raw_amount_i = total_value_minor × percentage_i / 100  
rounded_amount_i = round(raw_amount_i)
```

The system must allocate any rounding remainder deterministically, preferably to the largest line or an explicitly selected rounding recipient, and show the result before approval.

Fixed calculation

```
sum(fixed_amount_i) ≤ total_value_minor  
unallocated_amount = total_value_minor - sum(fixed_amount_i)
```

The unallocated amount should be displayed clearly. It must not silently disappear.

Calculation preview

Before approval, show:

- project value;
- split mode;
- each member's percentage/fixed value;
- calculated amount;

- total allocated;
- unallocated amount;
- validation errors;
- rounding note, if any.

The exact calculation algorithm should have unit tests covering zero values, decimals, rounding, duplicate members, invalid totals, and currency changes.

8. Screens and user experience

MVP navigation

1. Dashboard

- total owed;
- total paid this period;
- outstanding owed by person;
- projects awaiting approval;
- overdue tasks;
- recent activity.

2. Projects

- search, filter, sort, status badges;
- project creation;
- project detail workspace.

3. Project workspace

- overview;
- team and compensation;
- milestones/tasks;
- approval panel;
- payout summary;
- audit history.

4. Payout ledger

- owed, partially paid, paid, disputed;

- person/project/date filters;
- payment entry form;
- CSV export.

5. **Team** (Admin/PM)

- members, roles, active status, payout contact reference.

6. **My work**

- assigned tasks and personal payout history for team members.

My Work: how team members know what they have done

The My Work page is a first-class MVP feature, not an optional dashboard. It should answer three questions immediately: **What is assigned to me? What have I completed? What am I owed?**

The page should contain:

Area	Information shown
Summary cards	Assigned tasks, completed tasks, in-progress tasks, blocked tasks, overdue tasks, completion percentage
Current projects	Project name, my role, project progress, my task progress, health status, target date
My tasks	Task, project, status, due date, completion date, blocker indicator
Completed work	Completed tasks and milestones, completion notes, links/evidence, timestamps
My payouts	Total owed, total paid, remaining balance, payout status by project
Activity timeline	Assignments, status changes, completed work, comments, approvals, and payment events involving the member

Use these task statuses:

Not Started → In Progress → Done
↳ Blocked

Waived

When a member marks a task **Done**, require a completion note and completion date. An optional link or attachment may be added as evidence. When a task is marked **Blocked**, require the blocker reason and what is needed to continue. A member should not have to calculate progress manually: project progress should be derived from completed required tasks divided by total required tasks, while the member's personal progress should be derived from their own assigned tasks.

Display a separate project health indicator: **On Track**, **At Risk**, **Blocked**, or **Overdue**. Health is different from lifecycle status; a project can be **In Progress** and **At Risk** at the same time. Completing a task never creates a payout obligation by itself. The payout becomes **Owed** only after project approval.

The personal contribution history should allow a member to review past projects, roles, completed tasks, completed milestones, completion dates, amounts owed, amounts paid, and outstanding balances. This becomes the team's shared evidence when reviewing progress or resolving payout questions.

In-app notifications for progress

The MVP should notify members when they receive a task, when a task is due soon or overdue, when a project is returned for changes, when a project is approved, when a payout is created, and when a payment is recorded. Start with in-app notifications; add email or WhatsApp only after the workflow is being used consistently.

UX principles

- Make the current state and next action obvious.
 - Display money values with currency everywhere.
 - Use confirmation dialogs for approval, reopening, voiding, and adjustments.
 - Require reasons for rejection, reopening, adjustments, and write-offs.
 - Never hide why a payout is locked or why an action is unavailable.
 - Use non-destructive editing and clear audit history instead of silent overwrites.
-

9. Security and data protection

1. Require authenticated access for every application route and API action.
 2. Use PostgreSQL Row Level Security policies as a second authorization layer.
 3. Do not expose service-role database keys in the browser.
 4. Encrypt data in transit and rely on managed database encryption at rest.
 5. Minimize stored payout contact data; store only what is needed for manual reconciliation.
 6. Mask phone numbers in ordinary list views where practical.
 7. Never put payment references, secrets, or personal data in client-side logs.
 8. Add rate limits to login and sensitive mutation endpoints.
 9. Keep an audit trail for role changes and administrative access.
 10. Configure daily database backups and document restoration steps.
 11. Add a retention policy for uploaded payment evidence.
 12. Use environment variables for secrets and rotate credentials if a team member leaves.
-

10. Reporting and reconciliation

Required reports

- Total outstanding owed by member.
- Total owed by project.
- Total paid by date range.
- Projects completed but not fully paid.
- Partial payments.
- Adjustments and write-offs.
- Overdue projects and tasks.
- Internal versus external project payout totals.

CSV export fields

- Project code and name
- Client type/client name

- Project completion date
- Recipient
- Role on project
- Original amount owed
- Adjustments
- Amount paid
- Remaining balance
- Payout status
- Payment references and dates
- Approver

The export should be suitable for review in Ledgerio or a bookkeeping spreadsheet without claiming to be a full accounting integration.

11. API and module boundary suggestion

The application can use Next.js route handlers or server actions, but business rules should be kept in service modules rather than embedded in page components.

```
src/  
  app/  
    (authenticated)/  
      dashboard/  
      projects/  
      ledger/  
      team/  
      my-work/  
    api/  
  modules/  
    auth/  
    projects/  
    compensation/  
    tasks/  
    approvals/  
    payouts/  
    payments/  
    audit/  
    reporting/  
  lib/  
    db/  
    permissions/  
    validation/  
    money/  
    dates/  
  components/  
  tests/  
    unit/  
    integration/  
    e2e/
```

Important service functions

- `validateCompensationPlan(projectId)`
 - `previewCompensation(projectId)`
 - `requestProjectApproval(projectId, actorId)`
 - `approveProject(projectId, actorId)`
 - `rejectProject(projectId, actorId, reason)`
 - `recordPayment(ledgerEntryId, paymentInput, actorId)`
-

- `createAdjustment(ledgerEntryId, adjustmentInput, actorId)`
- `reopenProject(projectId, reason, actorId)`
- `getOutstandingByMember(filters)`

These functions should be transaction-aware and independently testable.

12. Delivery roadmap

The phases below are implementation phases, not just feature lists. Each phase should end with a usable increment, a database migration, automated checks, and a short review with the AGOD team before the next phase begins.

Phase 0 — Decisions and setup

- Confirm whether AGOD will reuse Ledgerio authentication.
- Confirm the authoritative operating timezone.
- Confirm whether every project is GHS-only for MVP.
- Confirm who may record payments.
- Define the initial team member and role list.
- Create the Supabase project, GitHub repository, environments, and migration workflow.

How to implement: Hold a short requirements workshop, write the five decisions into the repository README, create development/staging/production environments, and seed only test users in development. Define the operating timezone, GHS money rules, role permissions, and the initial project status diagram before building screens. The output is an approved baseline and an empty application that can run locally and in staging.

Exit condition: A developer can install the project, run migrations, sign in with a test account, and deploy the application to staging.

Phase 1 — Foundation

- Next.js/TypeScript application shell.
- Authentication and session handling.
- Role model and authorization helpers.
- Database schema and migrations.
- Audit event infrastructure.

- CI checks and error monitoring.

How to implement: Build authentication first, then a server-side permission service such as `can(actor, action, resource)`. Create the relational schema through versioned migrations. Add Row Level Security policies after the application permissions are working. Add a reusable audit-event service and record role changes, project mutations, and all future financial actions through it. Set up formatting, type checking, unit tests, preview deployments, Sentry, and database backup settings.

Exit condition: Users can log in, see only the routes and records allowed by their role, and every protected mutation is rejected correctly when the actor lacks permission.

Phase 2 — Projects and work tracking

- Project CRUD before approval.
- Assignments and compensation plan.
- Split preview and validation.
- Milestones and tasks.
- Project status transitions.

How to implement: Build the project workspace before the dashboard. Implement project creation, assignments, split validation, milestones, tasks, task notes, `Blocked`, and `Waived` states. Add the My Work page at the same time so members can see assigned, completed, overdue, and blocked work from the first pilot. Calculate project and personal progress from tasks rather than allowing arbitrary progress percentages. Add activity timeline entries for assignments and task status changes.

Exit condition: A team member can sign in, see their assignments, update their own tasks, record completion evidence or a blocker, and verify that their My Work progress matches the project workspace. A PM can create a project and see the same changes.

Phase 3 — Approval and ledger

- Request approval.
- PM/Admin approve/reject.
- Atomic snapshot and ledger generation.
- Locking after approval.
- Ledger views and personal payout history.

How to implement: Add the `PENDING_APPROVAL` request action, approval/rejection forms, required rejection reasons, compensation preview, and the approval transaction. The transaction must lock the project, validate required tasks and splits, create the immutable compensation snapshot, create one payout ledger entry per eligible recipient, append audit events, and set the project to `COMPLETED`. Add idempotency protection so retries cannot create duplicate ledger entries. Lock ordinary edits after approval and make reopening an Admin-only action with a required reason.

Exit condition: A complete project can be approved once and only once, the payout calculations reconcile exactly to the approved snapshot, and a team member can see the resulting payout in My Work without being able to alter it.

Phase 4 — Manual payments and reporting

- Payment transactions and partial payment support.
- Payment references and optional evidence upload.
- Reconciliation views.
- Dashboard and CSV export.
- Backup/restore runbook.

How to implement: Add payment transactions rather than a single paid flag. Allow authorized users to record full or partial payments with method, reference, date, notes, and optional evidence. Derive `OWED`, `PARTIALLY_PAID`, and `PAID` from the ledger balance. Add dashboards for total owed, paid, remaining balances, overdue work, and projects awaiting approval. Build CSV export and compare its totals with the dashboard and database queries. Document how to restore a backup before real financial history is entered.

Exit condition: AGOD can record a real manual payment, see the remaining balance immediately, export the ledger, and reconcile the export to the dashboard.

Phase 5 — Stabilization

- Seed realistic internal data.
- Run a pilot with two or three projects.
- Compare outputs against the existing spreadsheet/manual process.
- Resolve disputes and edge cases.
- Only then consider notifications, Ledgerio export, or automated payments.

How to implement: Run two or three representative projects through the entire workflow, including at least one rejection, one partial payment, one blocked task, and one adjustment scenario. Compare the system's amounts with the current manual process. Review the audit trail with the people who approve and pay contributors. Fix confusing screens and calculation defects before adding integrations. Treat automated payment as a separate project requiring security, retry, reconciliation, and approval design.

Exit condition: The team can explain every amount in the ledger, reproduce every status transition from the audit trail, and complete a backup/restore exercise. Only after this sign-off should the system be treated as the operational source of truth for payout obligations.

13. Testing and acceptance criteria

High-value automated tests

- A team member cannot approve a project.
- A PM/Admin can approve only a project in `PENDING_APPROVAL`.
- Approval creates exactly one snapshot and one ledger line per eligible recipient.
- Repeating the approval request cannot duplicate ledger entries.
- Percentage splits that do not total 100% are rejected.
- Fixed splits above the project value are rejected.
- Rounding is deterministic and totals reconcile.
- A payment cannot exceed the outstanding balance without an approved adjustment.
- Partial payment produces `PARTIALLY_PAID`.
- Final payment produces `PAID`.
- A normal user cannot edit an approved snapshot.
- Reopening requires Admin permission and a reason.
- Every material mutation produces an audit event.
- Historical records remain visible after a member becomes inactive.

Pilot acceptance test

AGOD should run at least three real or representative projects through the system and compare:

- calculated recipient amounts;

- approval history;
 - payment balances;
 - dashboard totals;
 - CSV export totals;
 - ability to explain every change from the audit log.
-

14. Additions worth considering after MVP

Prioritize additions by operational value, not novelty:

High value

1. **Approval reminders and overdue alerts** in-app, then email.
2. **Saved compensation templates** for recurring project types.
3. **Project profitability view**: project value, payout commitment, and unallocated margin.
4. **Ledger export/import bridge to Ledgerio**.
5. **Payment receipt upload** with controlled access.
6. **Dispute workflow** where a member can raise a question without editing the ledger.
7. **Period close** so a month can be reviewed and locked.
8. **Dashboard trend reports** for owed, paid, and outstanding amounts.

Medium value

1. Project budgets and non-payout costs.
2. Dependencies and milestone health indicators.
3. Reusable task templates.
4. Team capacity and workload view.
5. Multiple currencies, if AGOD genuinely needs them.
6. Slack/WhatsApp/email notifications, subject to privacy and operational fit.

Defer until there is a proven need

1. Automated Mobile Money/bank transfers.
2. Complex approval chains.

3. Hourly billing and timesheets.
 4. Client portals.
 5. General-purpose accounting features.
 6. Microservices or event-driven infrastructure.
-

15. Final recommendation

Proceed with a **Next.js/TypeScript + Supabase PostgreSQL/Auth + Tailwind/shadcn + GitHub Actions** modular monolith. The highest-risk part is not the dashboard; it is the integrity of the compensation calculation and the approval-to-ledger transition.

Before writing UI code, finalize these five decisions:

1. Is the MVP GHS-only?
2. Will authentication be shared with Ledgio or separate through Supabase Auth?
3. Are mixed percentage/fixed compensation plans allowed, or should the MVP use one mode per project?
4. Who may record a manual payment?
5. Does an approved project ever need to be reopened, and if so, is Admin-only reopening acceptable?

If those decisions are made, the system can be built quickly without sacrificing the auditability and payout controls that make it trustworthy.